PyTorch Fundamentals

Besher Alkurdi

Table of contents

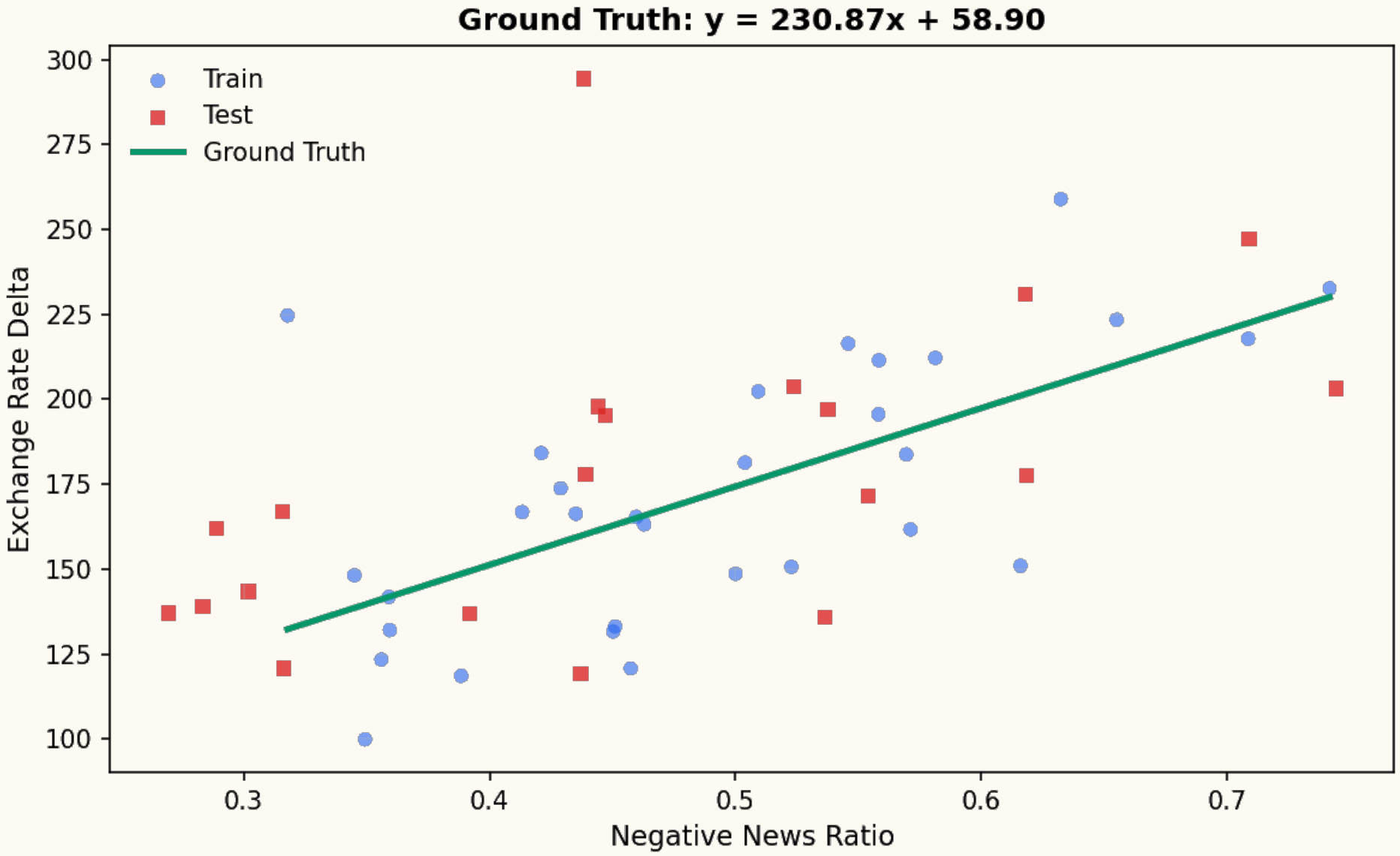
- How to Learn?
- PyTorch

How to Learn?

Predicting The Economy

Bad News Ratio	SYP-EUR Delta
0.15	75
0.42	130
0.68	160
0.91	200

What The Data Says



What is Our Error?

$$y - \hat{y} \text{ ??}$$

$$(y - \hat{y})^2$$

$$(y - (wx + b))^2$$

$$\frac{1}{n} \sum_{i=1}^{n} (y_i - (wx_i + b))^2$$

How can we decrease this?

This is a **loss function** and we want to *minimize* it. This is called **optimization**.

$(x = 0.5, y = 100)$, start with $w = 50$ and fix $b = 0$.

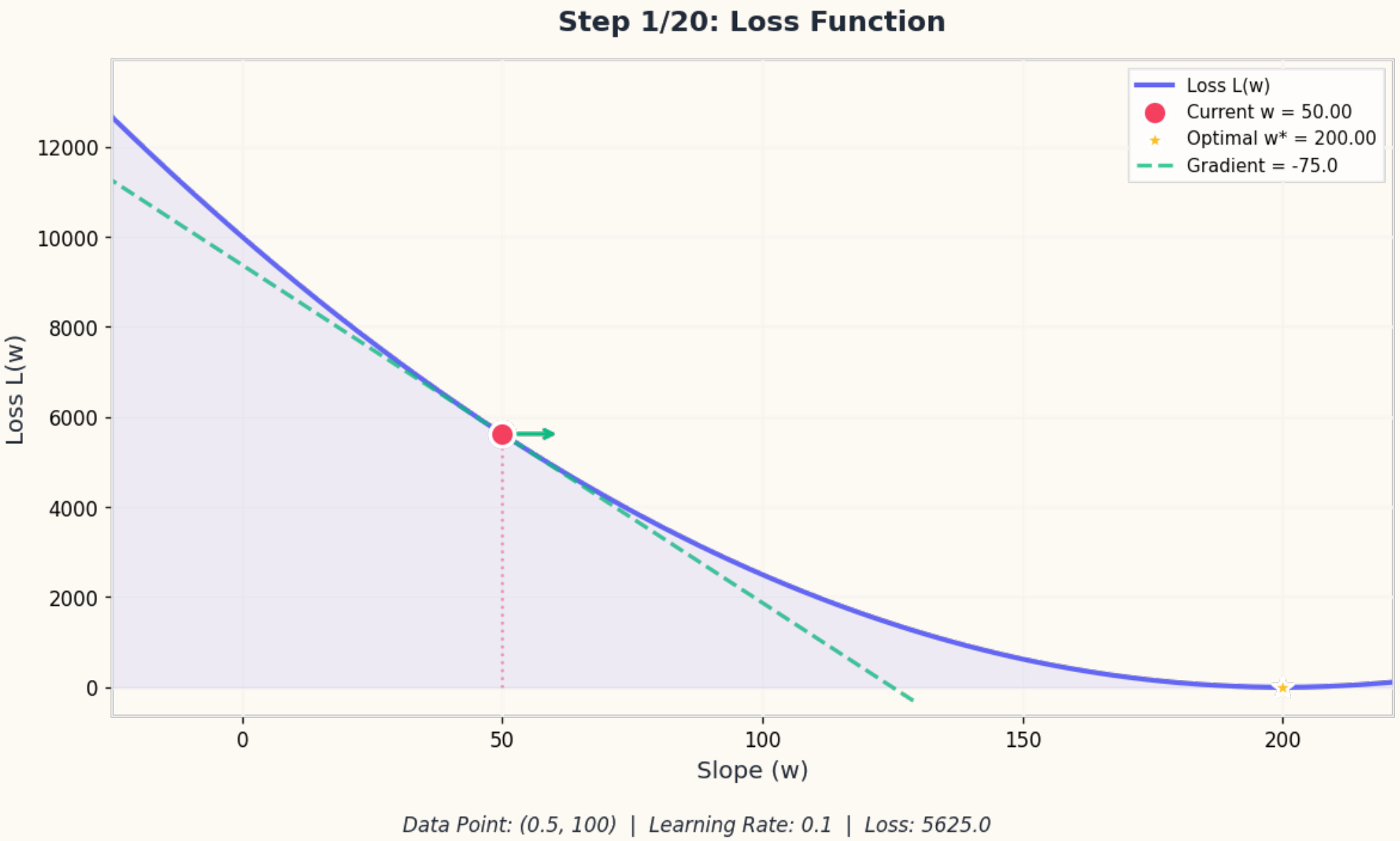
Our loss derivative w.r.t. w:

$$\frac{d}{dw}(y - (wx + b))^2 = -2x(y - (wx + b))$$

$$-2 \cdot 0.5 \cdot (100 - (w \cdot 0.5 + 0)) = -(100 - 0.5 \cdot w)$$

$$= (0.5 \cdot 50 - 100) = -75$$

Let’s Optimize!



Wait, but how?

$$w = w - \frac{d}{dw}(y - (wx + b))^2$$

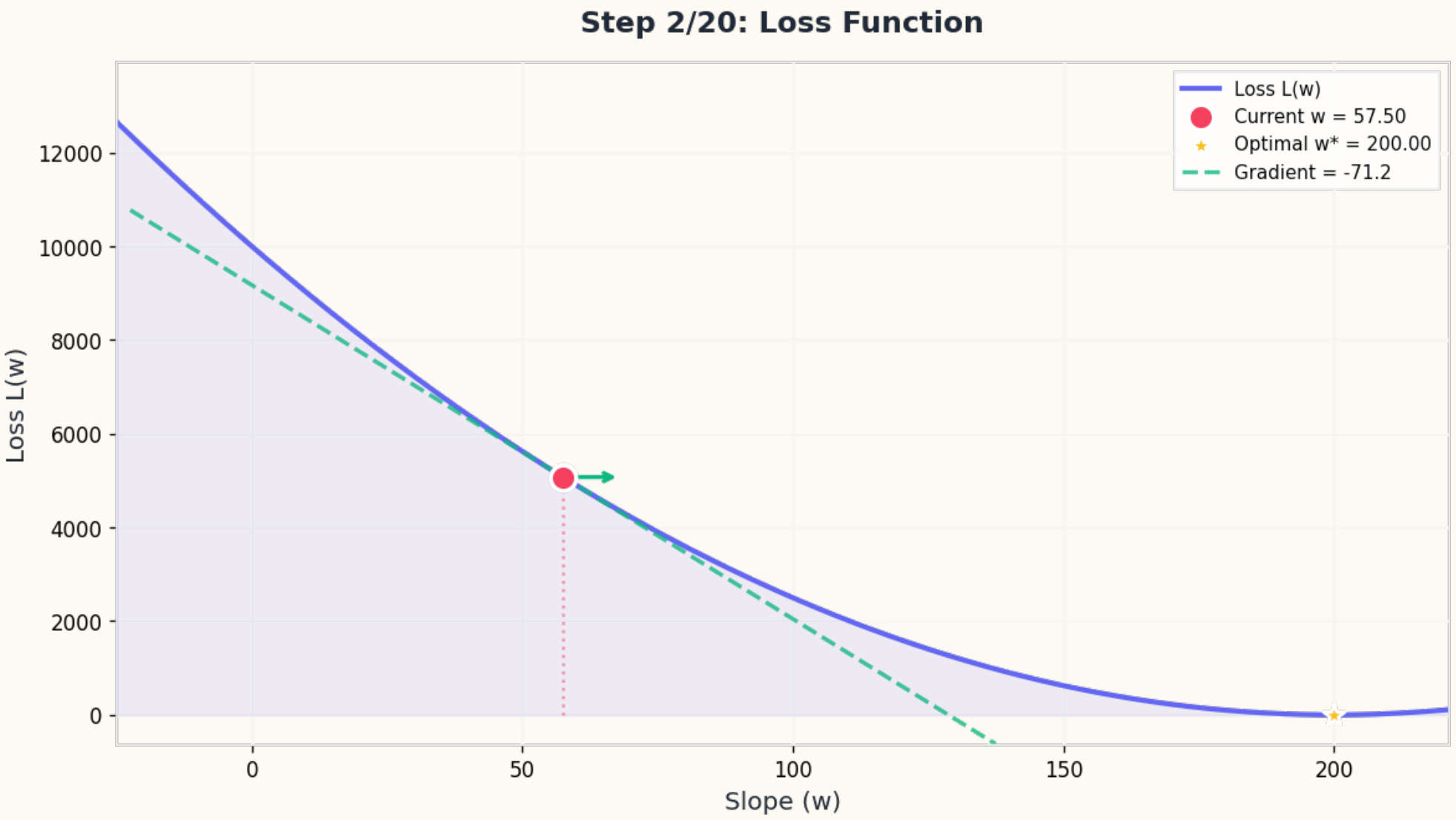
$$w = w - \nabla w$$

What if gradient is too big?

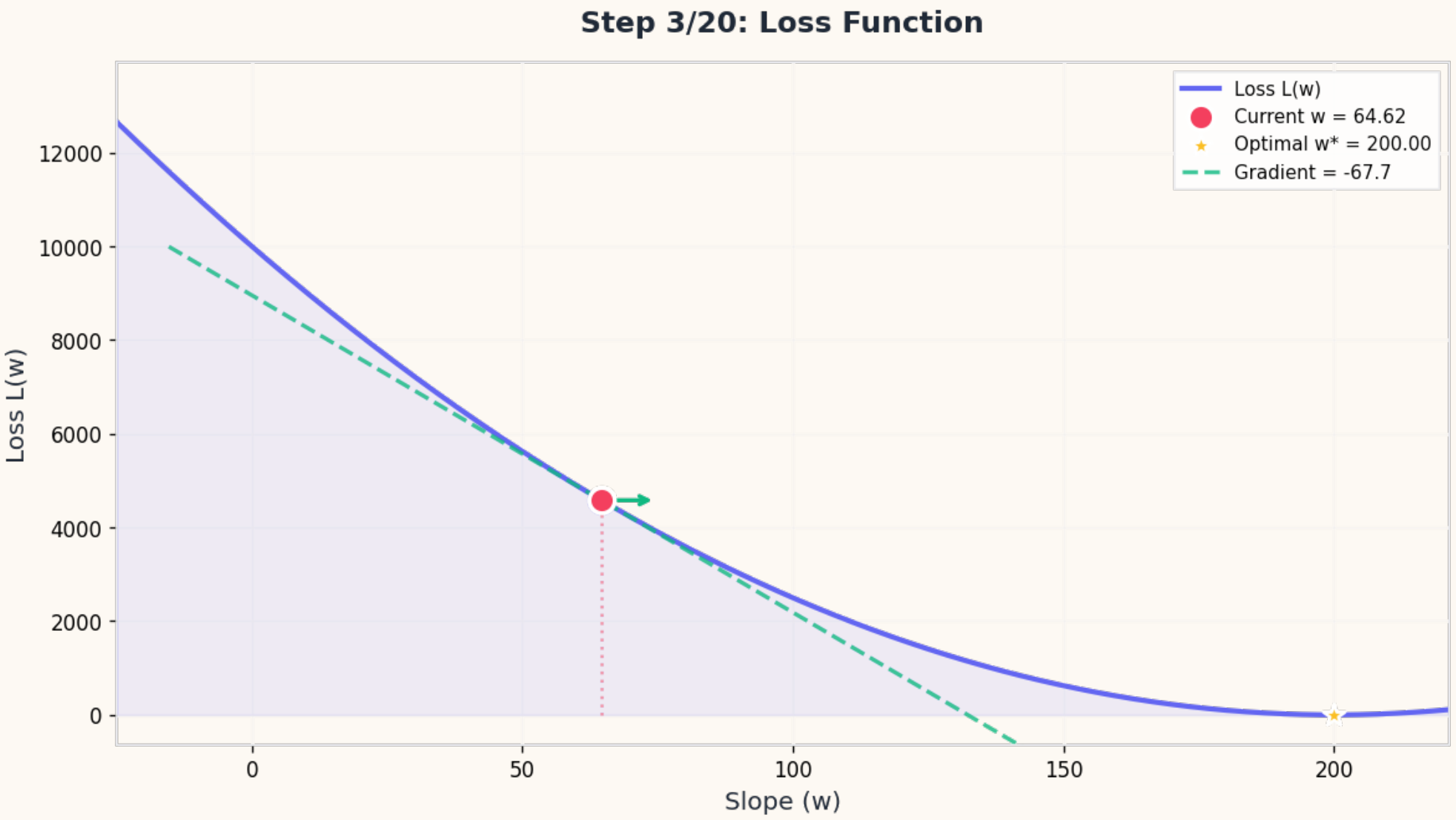
$$w = w - \alpha \cdot \nabla w$$

α is learning rate :)

Let’s Optimize!

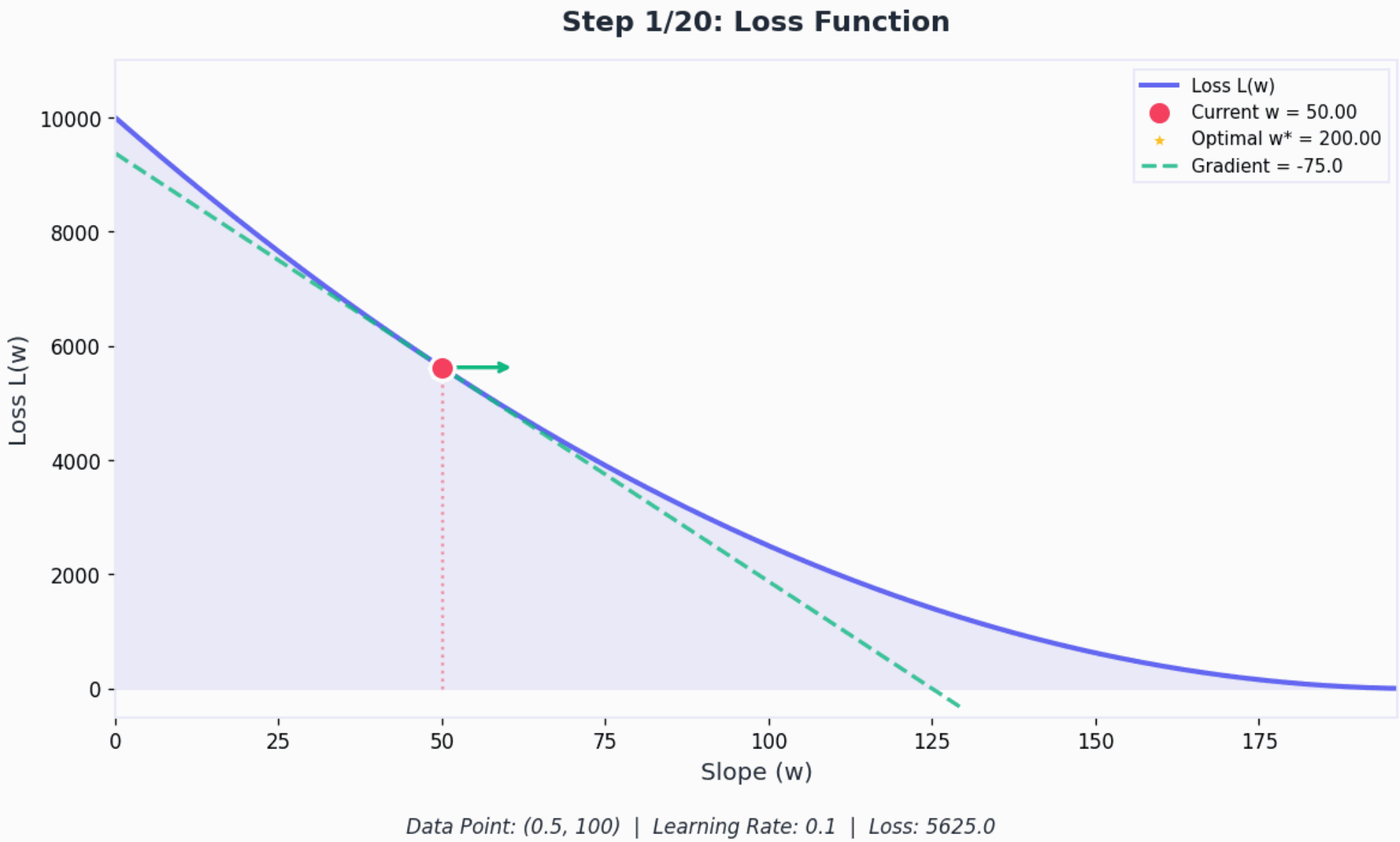


Let’s Optimize!

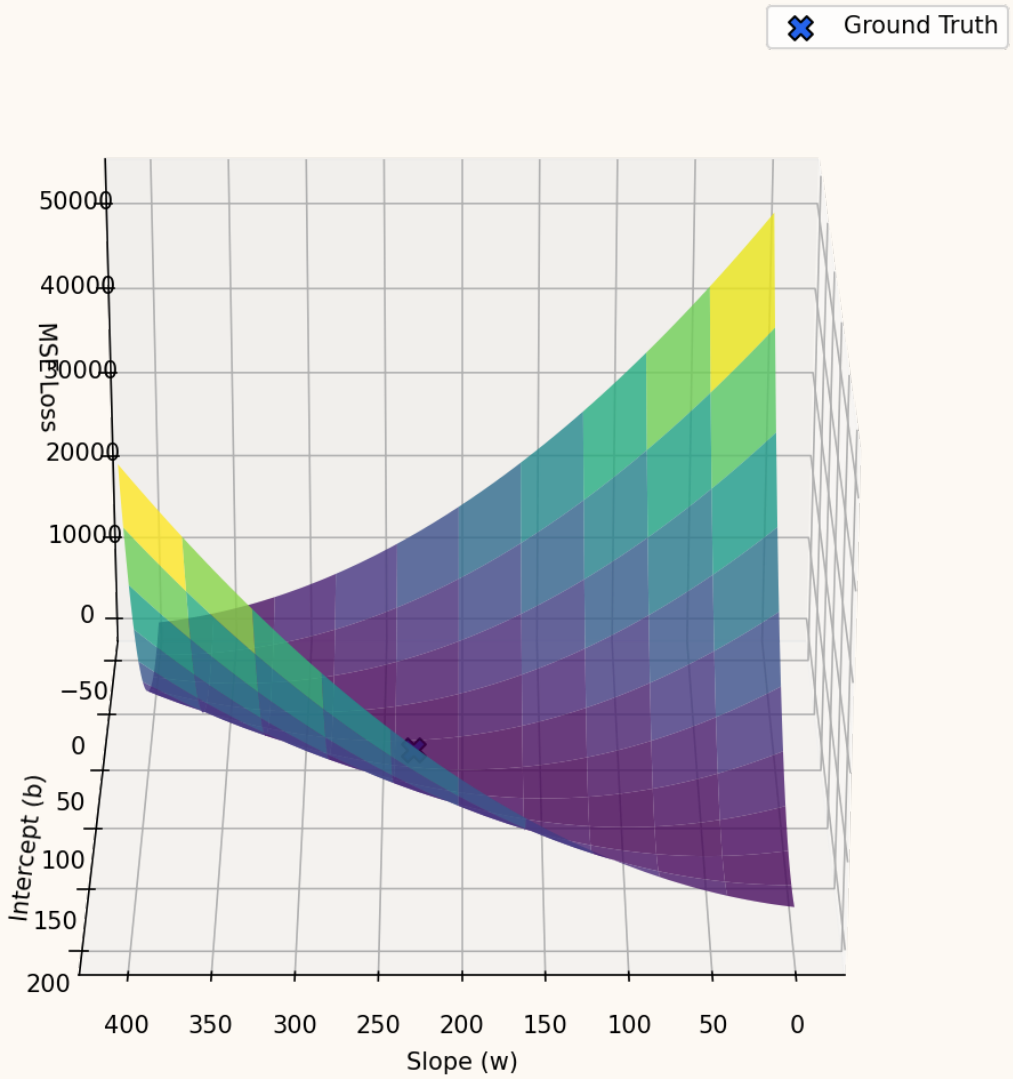


Data Point: (0.5, 100) | Learning Rate: 0.1 | Loss: 4581.6

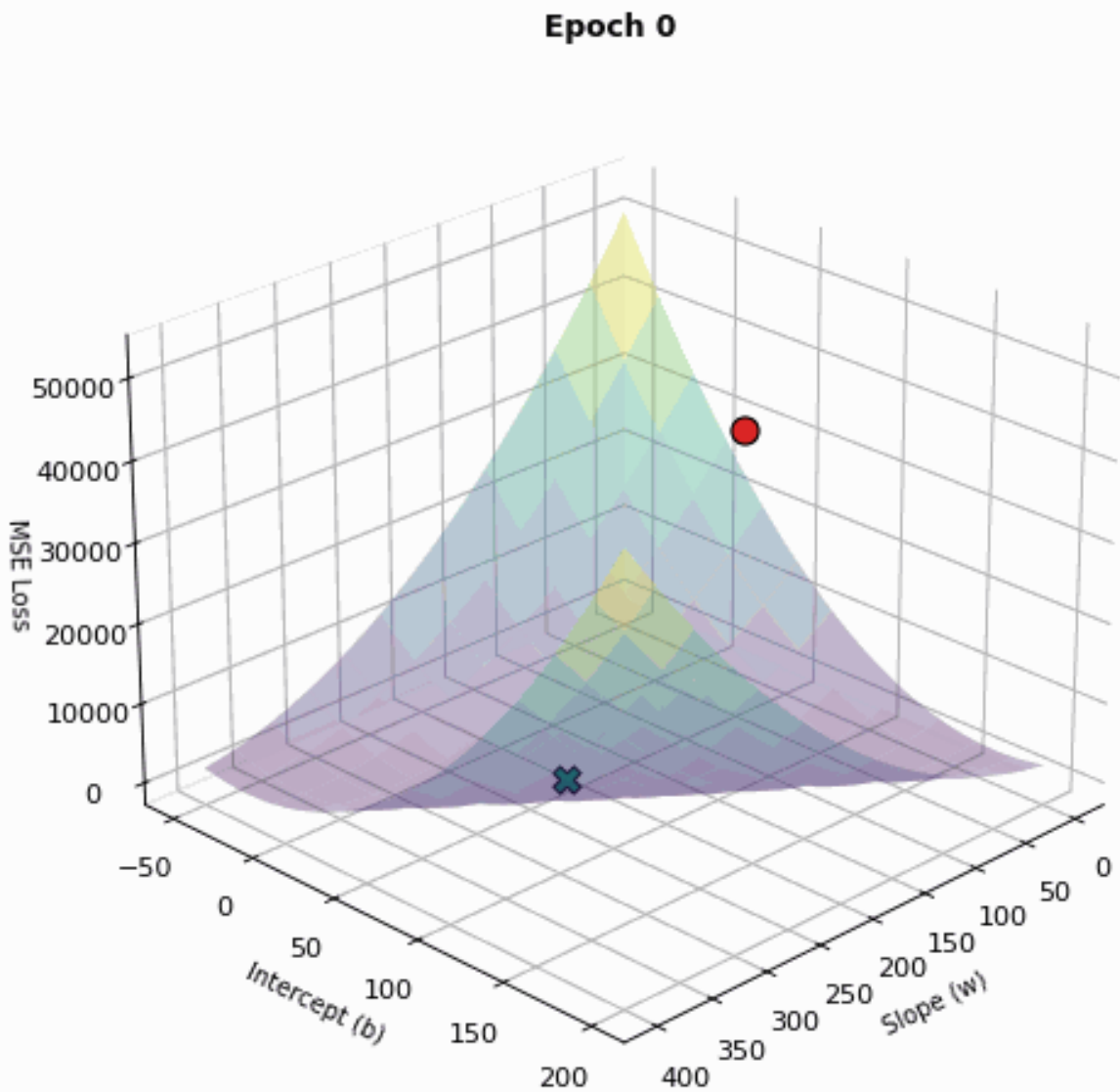
Let’s Optimize!



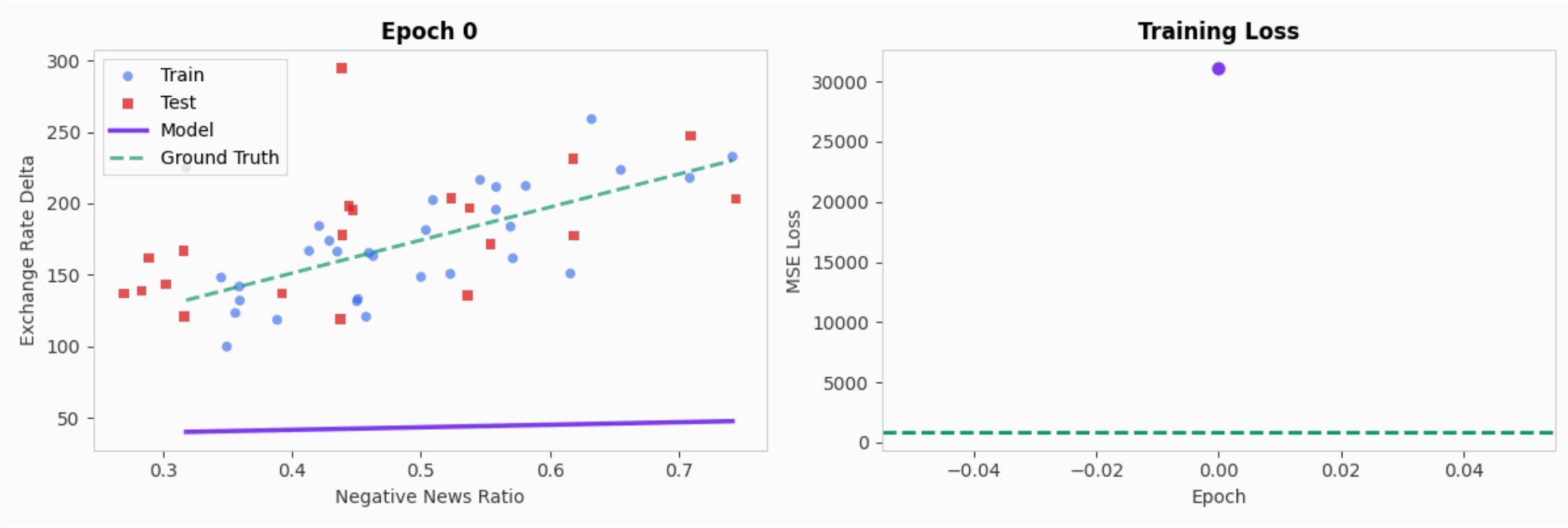
w isn't The Only Player



Once More, Let’s Optimize!

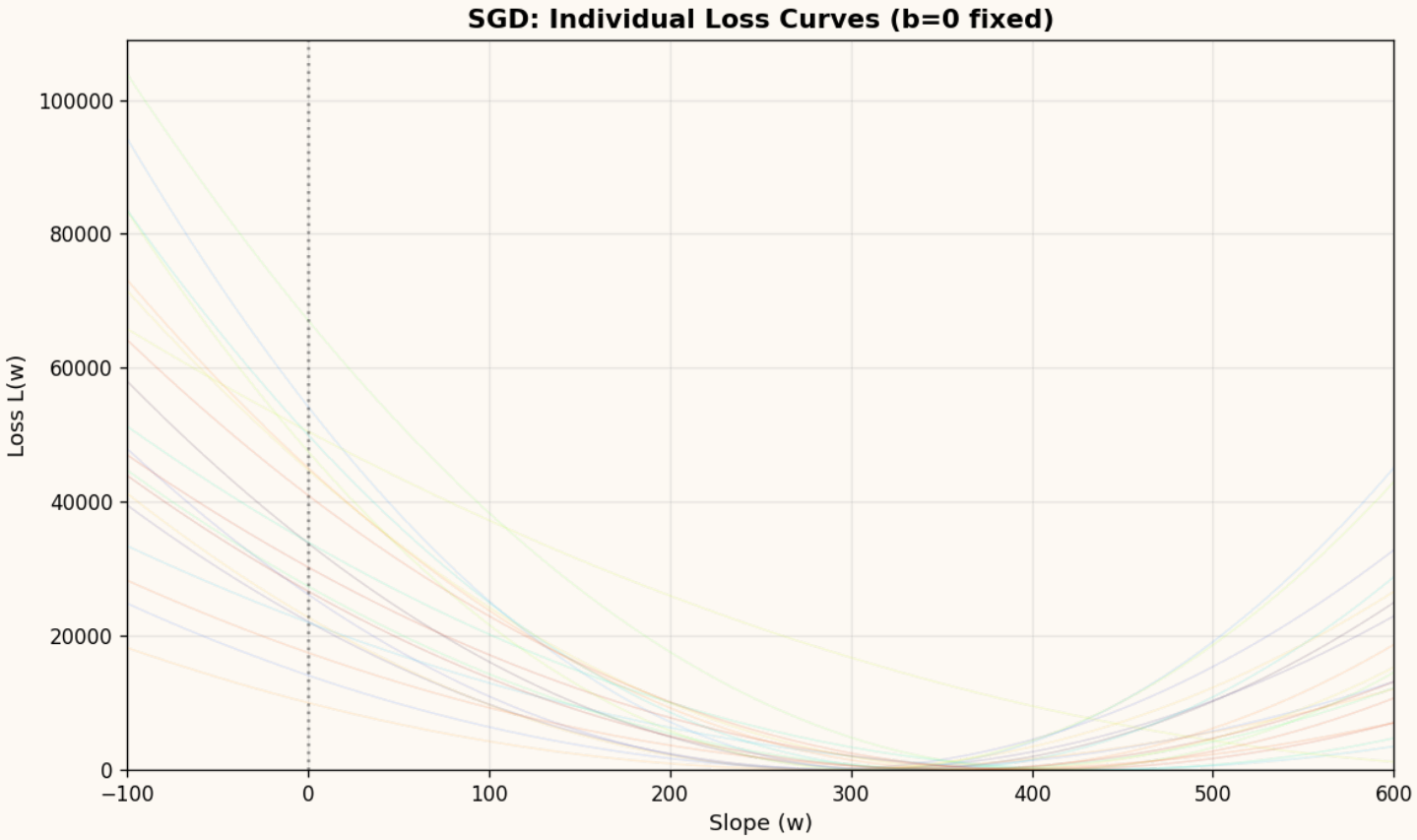


What Happens to Our Line?

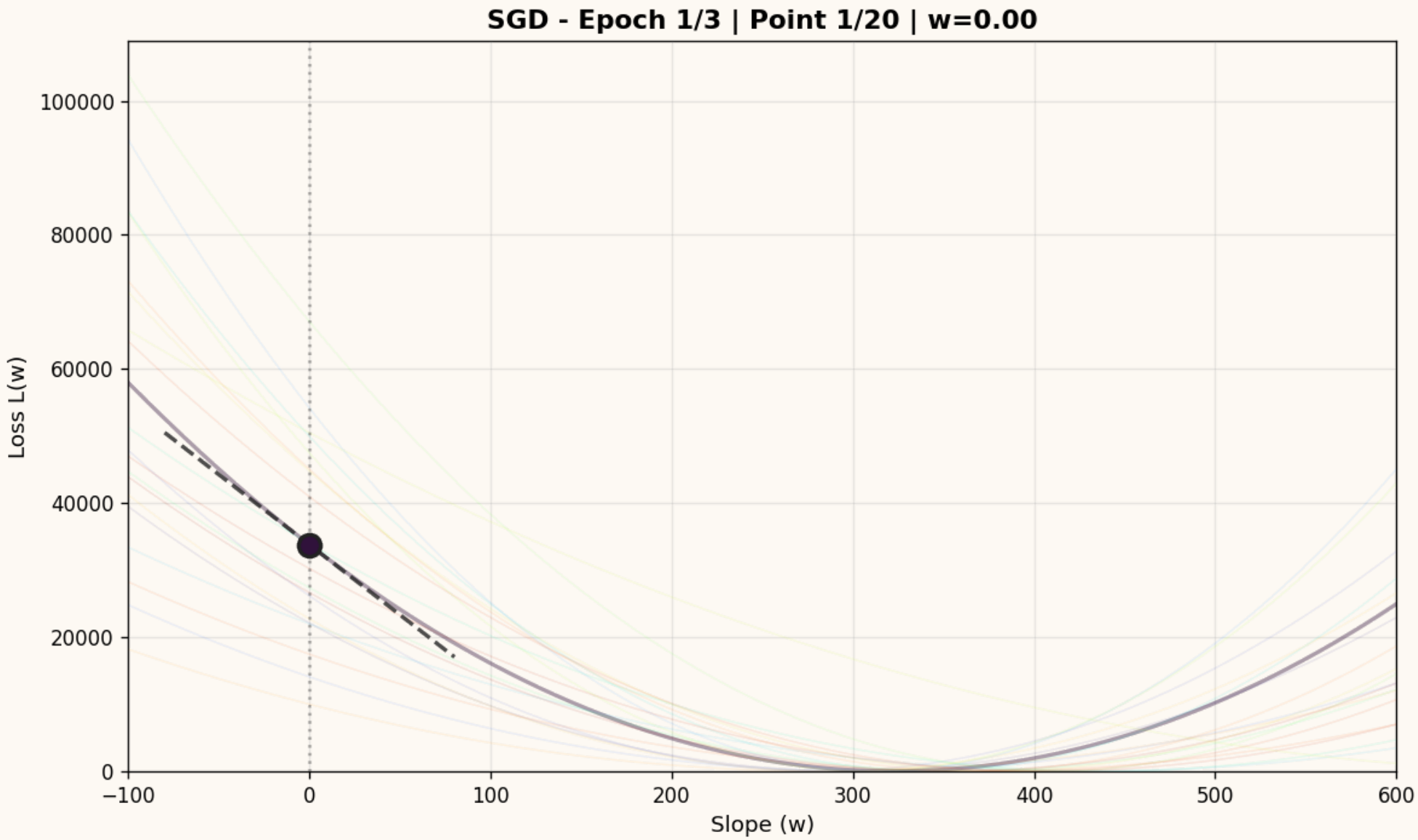


Memory is Limited

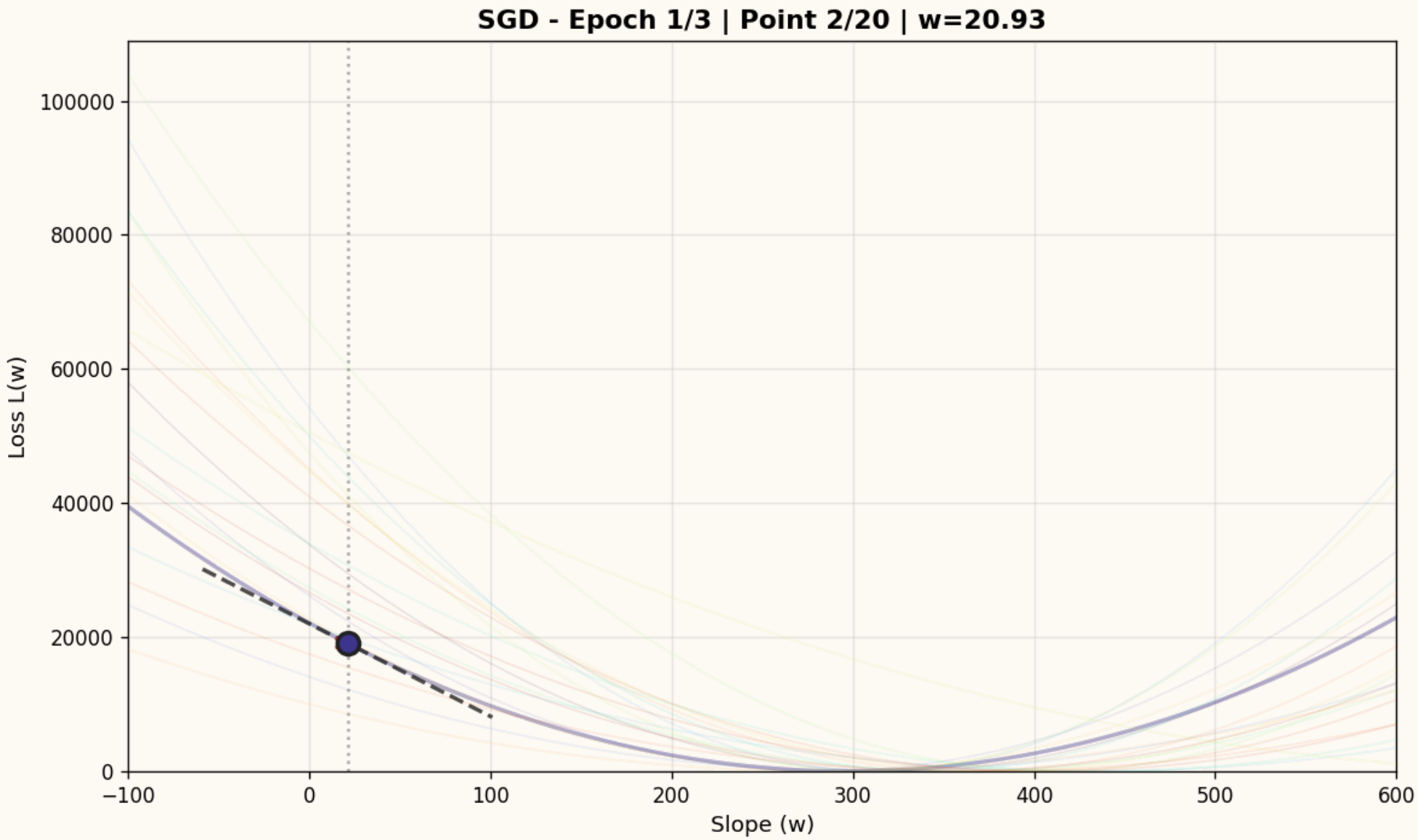
What we did until now is called **Gradient Descent**. Not all data fits in memory. So, we use **Stochastic Gradient Descent** (SGD).



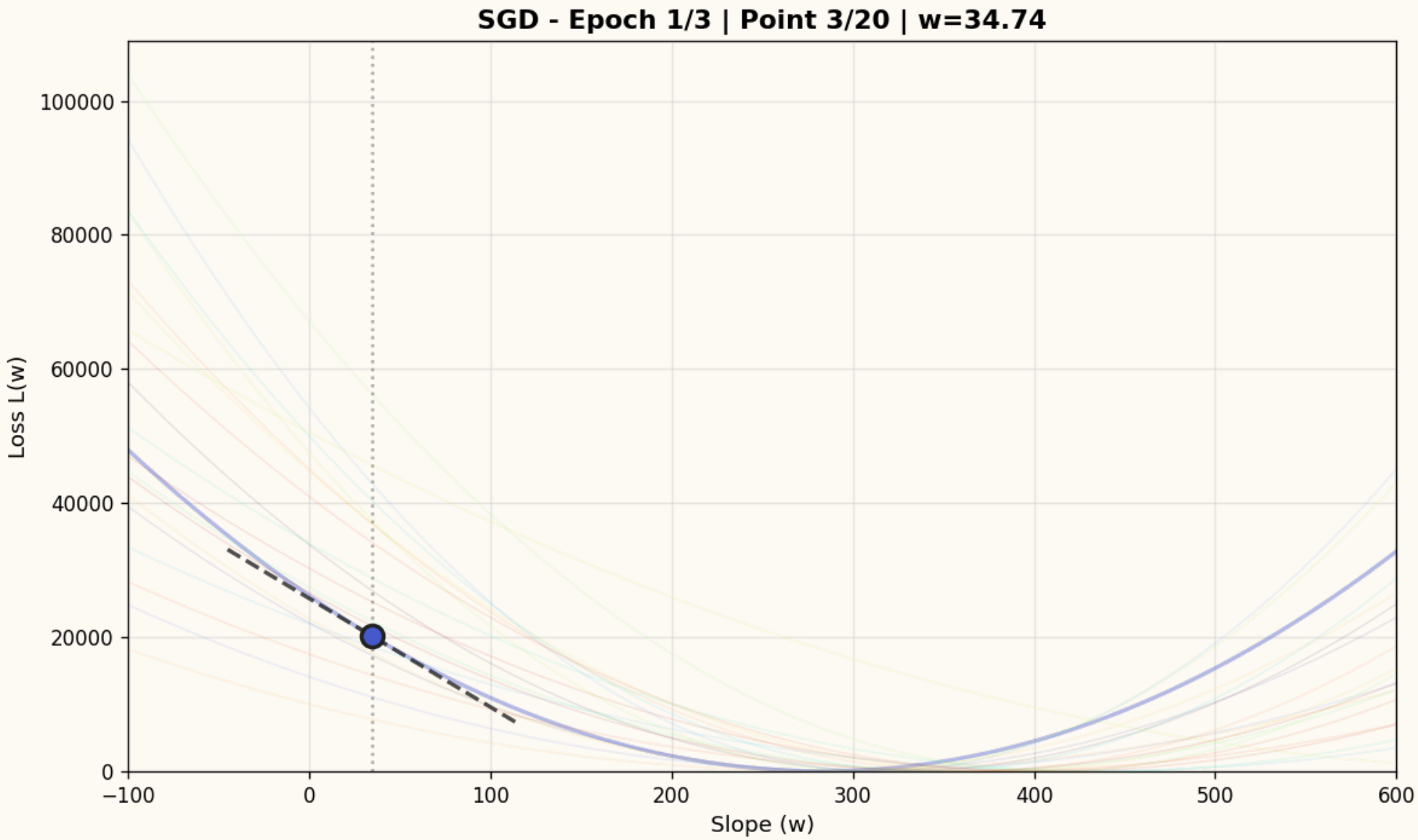
SGD


SGD - Epoch 1/3 | Point 1/20 | w=0.00
Loss L(w)
Slope (w)
100000
80000
60000
40000
20000
0
−100
0
100
200
300
400
500
600

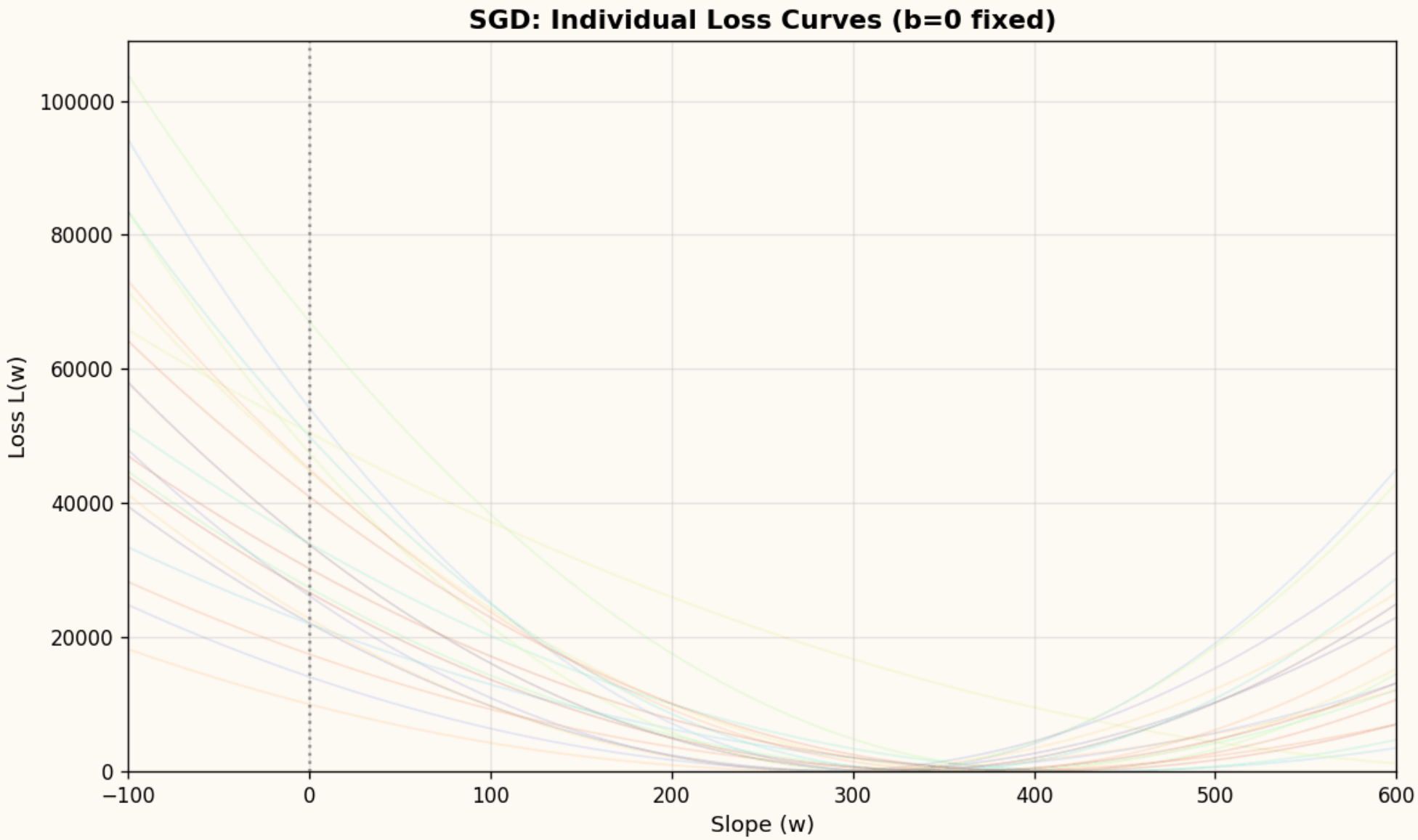
SGD


SGD - Epoch 1/3 | Point 2/20 | w=20.93
Loss L(w)
Slope (w)
100000
80000
60000
40000
20000
0
−100
0
100
200
300
400
500
600

SGD


SGD - Epoch 1/3 | Point 3/20 | w=34.74
Loss L(w)
Slope (w)
100000
80000
60000
40000
20000
0
−100
0
100
200
300
400
500
600

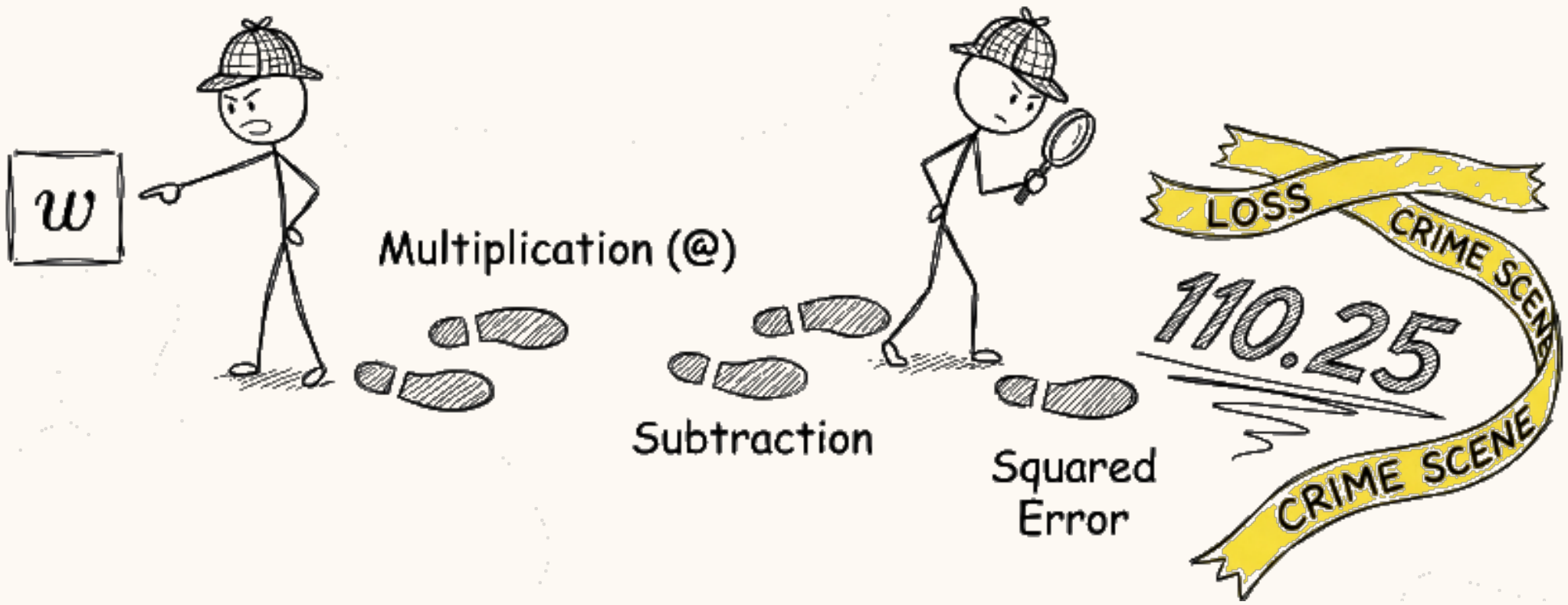
SGD


SGD: Individual Loss Curves (b=0 fixed)
Loss L(w)
Slope (w)
100000
80000
60000
40000
20000
0
−100
0
100
200
300
400
500
600

PyTorch

Life Isn’t That Simple

- Billions of parameters
- Chain of operations



Why PyTorch?

Tensors

- GPU acceleration
- Gradient tracking
- Memory-efficient operations

Autograd

- Automatic differentiation
- Dynamic computational graph

A Tensor is Memory + Metadata

```
x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])
```

```
MEMORY:     [1] [2] [3] [4] [5] [6]

METADATA:
  shape:   (2, 3)     ← 2 rows, 3 columns
  stride:  (3, 1)    ← jump 3 for rows, 1 for cols
  dtype:   float32   ← 4 bytes per element
  device:  cpu        ← where it lives
```

Now with PyTorch

```
x = torch.tensor([[0.5]])
y = torch.tensor([[100.0]])

# Parameters
w = torch.tensor([[50.0]], requires_grad=True)
b = torch.tensor([[0.0]], requires_grad=False)
```

```
y_pred = x @ w + b  # Matrix multiplication
loss = ((y_pred - y) ** 2).mean()
```

This was the **forward pass.**

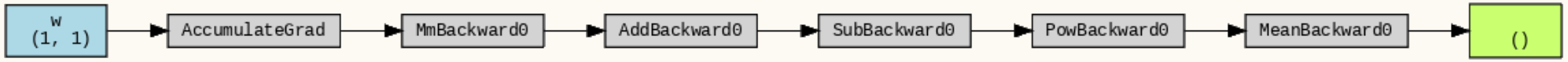
```
# Backward pass. ONE line!
loss.backward()
```

What Just Happened?

```
loss.backward()

print(w.grad)   # tensor([[-75.]])
print(b.grad)   # None
```

The Computational Graph



Gradient Descent Step by Step

```python
lr = 0.1

# 1. Forward
y_pred = x @ w + b
loss = ((y_pred - y) ** 2).mean()

# 2. Backward
loss.backward()

# 3. Update
with torch.no_grad():  # Why?
    w -= lr * w.grad
    b -= lr * b.grad

# 4. Zero gradients
    w.grad = None
    b.grad = None
```

The Full Training Loop

```python
for epoch in range(1000):
    # Forward
    y_pred = x @ w + b
    loss = ((y_pred - y) ** 2).mean()

    # Backward
    loss.backward()

    # Update
    with torch.no_grad():
        w -= 0.001 * w.grad
        # b -= 0.001 * b.grad
        w.grad = None
        # b.grad = None

print(f"w = {w.item():.2f}")
print(f"b = {b.item():.2f}")
```

Abstractions: `nn.Linear`

```
import torch.nn as nn

model = nn.Linear(1, 1)
```

What `nn.Linear` does:

```
y = x @ W.t() + b
```

nn.Module and forward()

```python
from torch import nn

class FFN(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear1 = nn.Linear(1, 1)
        self.linear2 = nn.Linear(1, 1)

    def forward(self, x):
        x = self.linear1(x)
        x = self.linear2(x)
        return x
```

Some Notes

If you define a list of modules, use `nn.ModuleList`:

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([
            nn.Linear(1, 1) for _ in range(3)
        ])
# ...
```

Register tensors them with `nn.Parameter`:

```
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.W = nn.Parameter(torch.randn(1, 1))
# ...
```

Built-in Loss Functions

Loss	When to Use	Formula
`mse_loss`	Regression	$(\hat{y} - y)^2$
`cross_entropy`	Classification	-log(softmax)

```python
import torch.nn.functional as F
from torch import nn

# All the same thing:
loss = ((y_pred - y) ** 2).mean()
loss = F.mse_loss(y_pred, y)
loss = nn.MSELoss()(y_pred, y)
```

Views vs Copies

```
x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

y = x[0, :]  # First row
y[0] = 999

print(x)  # [[999, 2, 3], [4, 5, 6]]
```

Try:

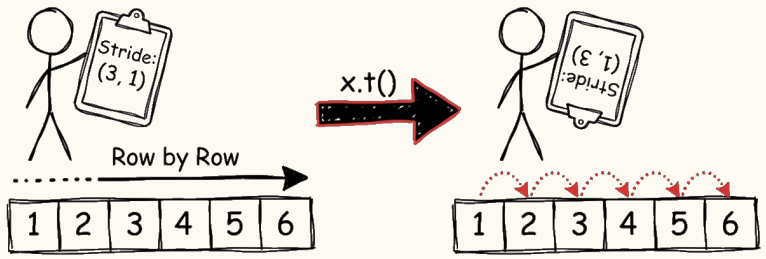
```
print(x.data_ptr() == y.data_ptr())
```

y shares memory with x!

To copy:

```
y = x[0, :].clone()
```

Transpose is a View




```python
x = torch.arange(6).reshape(2, 3)
y = x.T  # Transpose

print(x.stride())  # (3, 1)
print(y.stride())  # (1, 3)
print(x.data_ptr() == y.data_ptr())
```

No data movement! But…

```python
y.is_contiguous()
```

```python
z = y.contiguous()
print(x.data_ptr() == z.data_ptr())
```

Summary: The Training Loop

1. **Forward**: y_pred = model(x)
2. **Loss**: L = loss_fn(y_pred, y)
3. **Backward**: L.backward()
4. **Update**: optimizer.step()
5. **Zero**: optimizer.zero_grad()
6. **Repeat**

Summary

- Tensors = memory + metadata
- Gradients accumulate. Remembert to zero explicitly
- `requires_grad=True` enables autograd
- Views share memory; copies allocate new